

Viewing

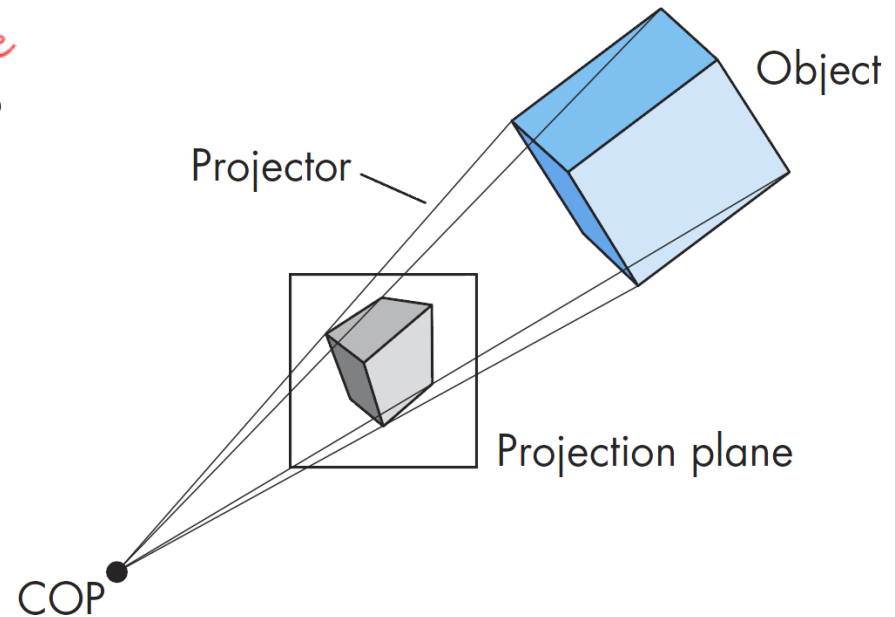
Computer Graphics
Instructor: Sungkil Lee

Today

- **Viewing elements**
- **~~Perspective and parallel projection~~**
- **Computer viewing**
 - Forward/Backward approaches
- **Moving Camera: building LookAt matrix**

Four Basic Elements of Viewing

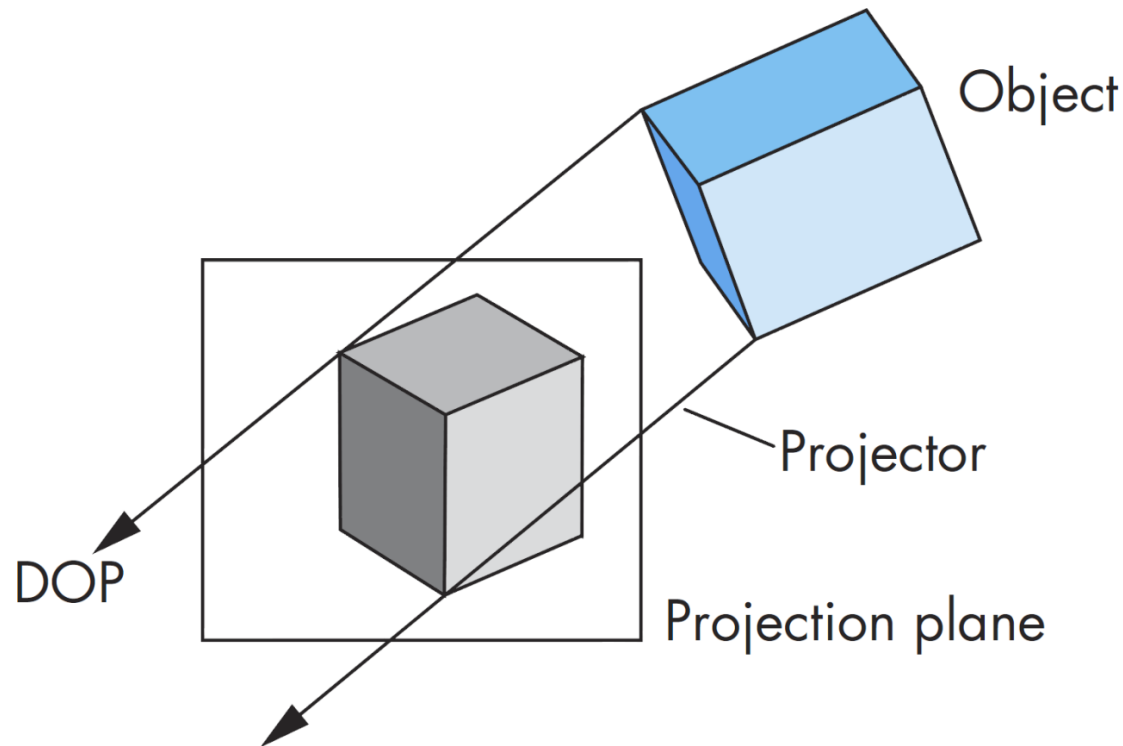
- **Objects** *point*
- **Viewer: center of projection (COP)**
 - Corresponds to the center of the lens in the viewer's camera or eyes
 - In CG, COP is the origin of the camera frame for perspective views
- **Projection surface:** *필름*
 - Standard projections project onto a **plane**
- **Projectors** *projection을 carry하는 line*
 - lines that either **converge** at COP or are **parallel**
 - Such projections **preserve lines**, but not necessarily angles



Parallel Projection

- **What if the COP goes to infinity?**

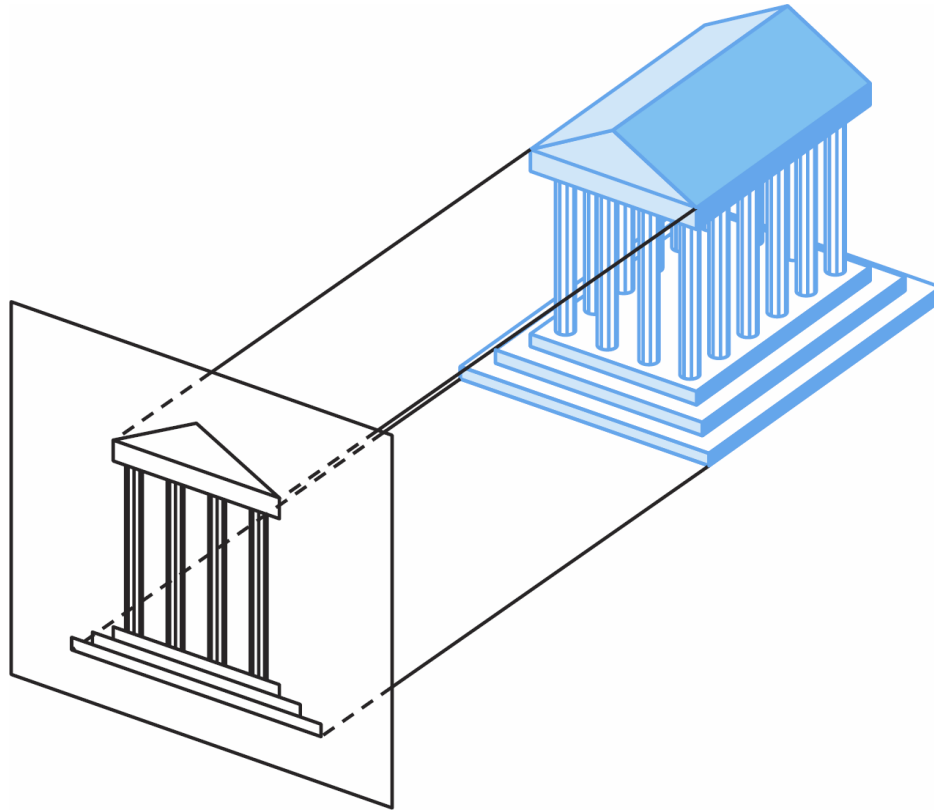
- The projectors get to be *parallel*, leading to parallel projection.
- Then, COP implies no more a point but the direction of projection (*DOP*).



Orthographic Projection

- **A special case of parallel projection**

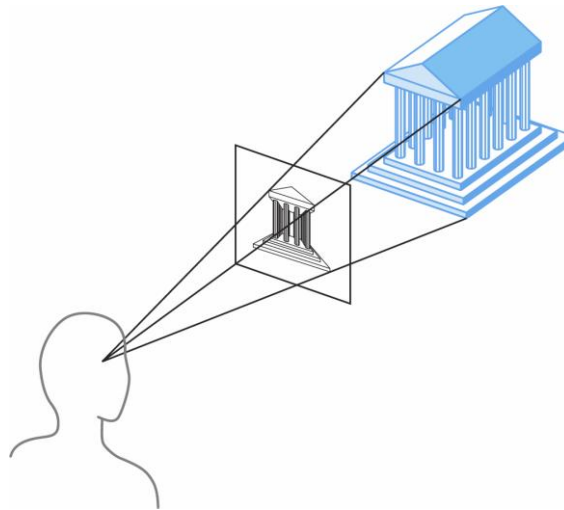
- Projectors are orthogonal to the projection surface.
- Usually, applied for 2D viewing



Perspective Viewing의 과정

• Perspective: **diminution** : 멀어질수록 작아진다

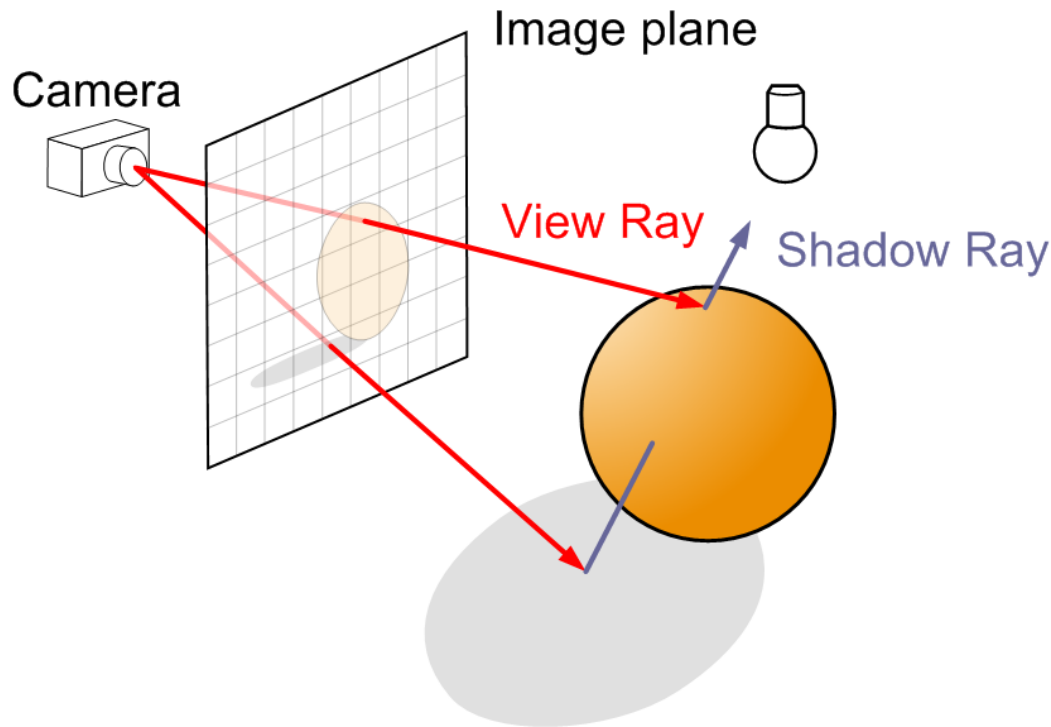
- When objects are moved farther from the viewer, their images in a projection surface become **smaller**.
- This size change provides natural realism; however, the amount of foreshortening is hard for us to measure.
 - **Foreshortening**: displaying objects closer (with a shorter depth) due to a different angle of vision (projection).
 - Primary applications are natural-looking images, rather than precise measurements.



Computer Viewing: Two Viewing Approaches

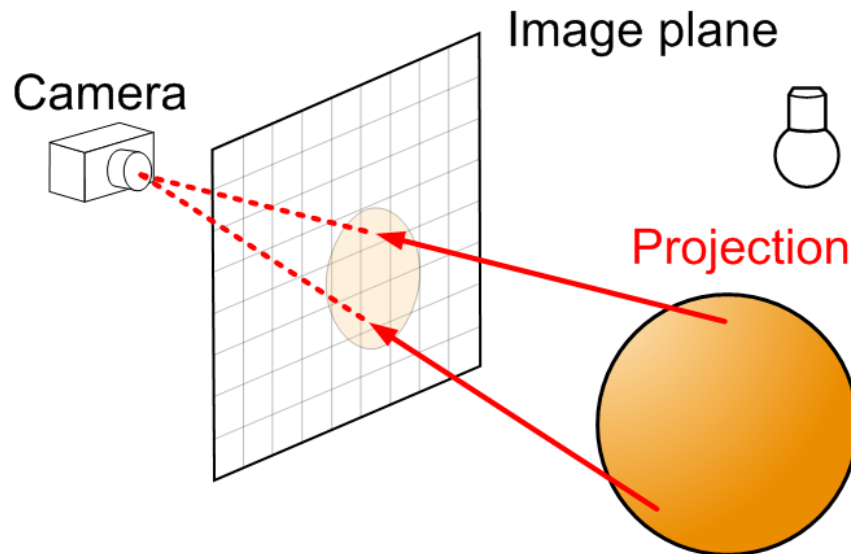
Viewing Approaches

- **Backward approach (ray tracing in CPU)**
 - Start from pixel
 - Explicitly construct the ray corresponding to the pixel
 - The ray that originates in camera, goes through the pixel, and intersects with the surface of some objects.
 - Ask what part of scene intersects with the ray



Viewing Approaches

- **Forward approach (pipeline approach)**
 - Naturally, a light ray comes into the image
 - Starting from a point in 3D, computes its projection into the image
 - Central tool is matrix multiplications
 - Combines seamlessly with model and view transformations



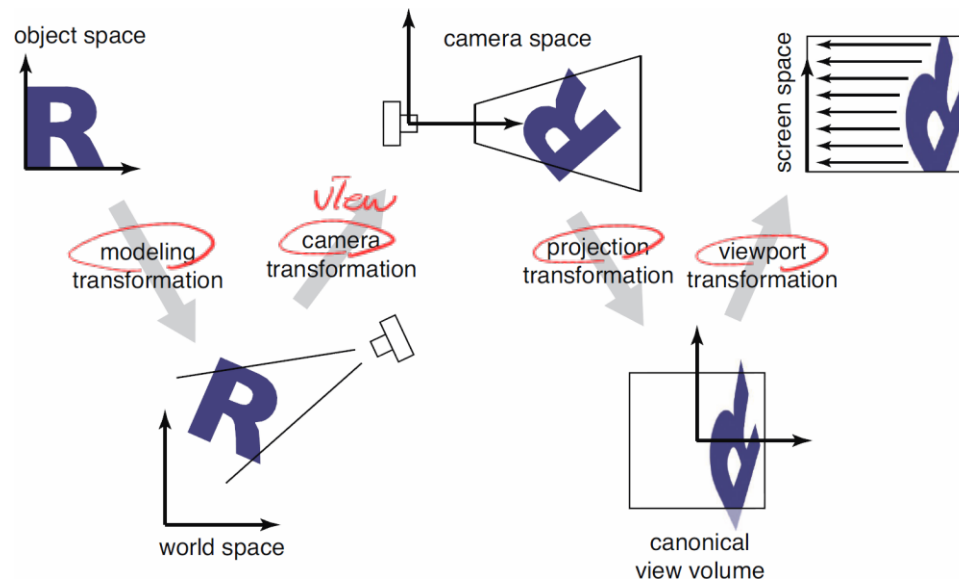
Forward Approach: Standard Pipeline

- **Four transformations**

- **Modeling:** Local (object) coords. → world coords.
- **View:** World coords. → camera coords.
- **Projection:** Camera coords. → normalized device coords. (NDC)
- **Viewport:** NDC → screen (window) coords.

- **Each transformation corresponds to a matrix multiplication.**

- Yielding *a concatenated matrix* to map a 3D point to its screen position.

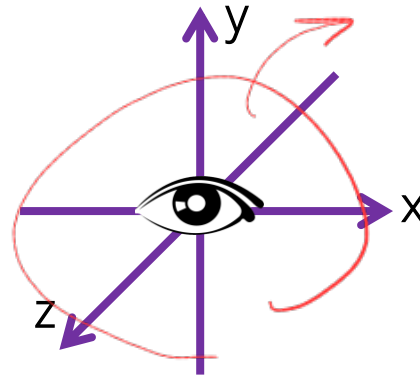
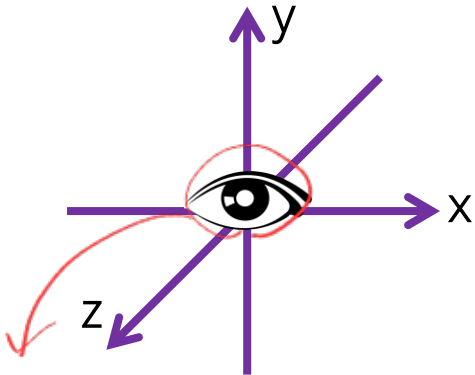


Moving Camera: Building LookAt Matrix

Model-View Duality

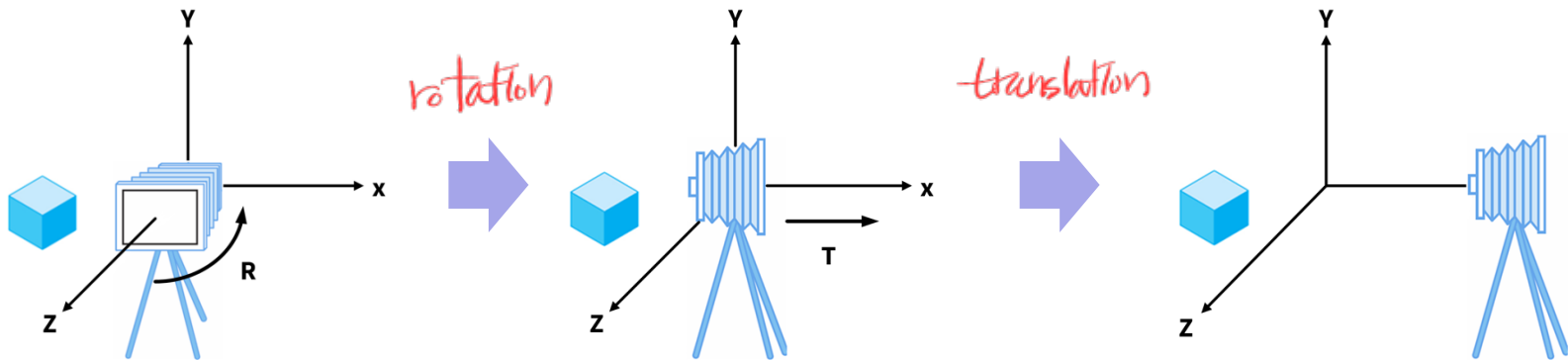
- **If we like to visualize an object, we can either**
 - move camera or move the object in the inverse direction
 - These two actions are actually equivalent (duality).
- **Model-view duality**
 - The camera could be understood as being fixed in the origin and the view on the scene is determined by the model-view transformation matrix.

→ 물체를 반대로 이동시킬
camera를 특정 위치로 이동시키는 것과 물체를 반대 방향으로
이동시키는 건 동일하다.



Moving Camera

- **We can move the camera to any desired position by a sequence of rotations and translations**
 - Since this way would be inconvenient, we prefer a more systematic way.



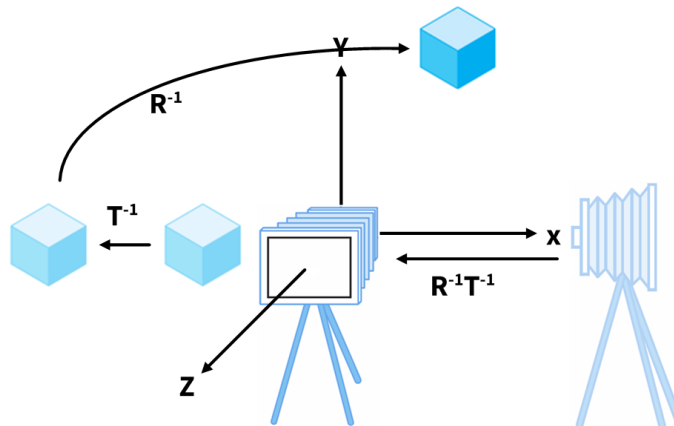
LookAt Method

- **LookAt()** method

위치와 방향을 지정해줌, camera는 origin이 고정됨

$\text{mat4 look_at}(\text{eye}, \text{at}, \text{up})$ camera의 위치 Vector (회전값)
camera의 위치
바라보는 좌표
look at matrix
View matrix

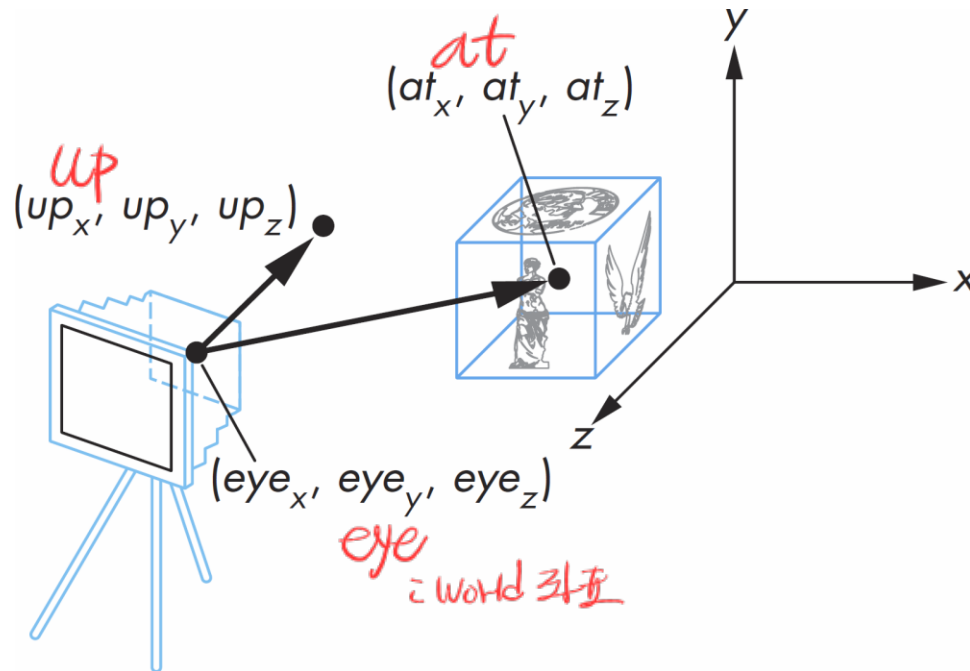
- a more standardized viewing mechanism.
- **Fix the camera** and move the objects using an inverse matrix multiplication.
- Hence, we are always in the camera frame (i.e., camera is in the origin).



LookAt Method

- **Viewing specification with (eye, at, up)**

- eye: a camera's location
- at: the center of the destination position to be viewed
- up: upward direction of the camera frame



LookAt Matrix

- eye, at, and up can define a camera **frame**, which has

- the origin at eye
- three basis vectors, u , v , and n are defined as:

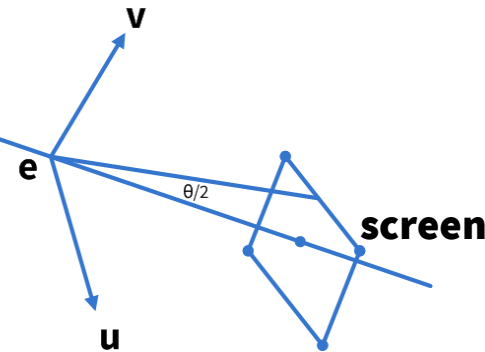
$$n = \text{normalize}(\text{eye} - \text{at})$$

scene에서 eye쪽으로 가다

$$u = \text{normalize}(\text{up} \times n)$$

$$v = \text{normalize}(n \times u)$$

- They are similar to x , y , z in the world space.



- Thus, the viewing transformation can be a **change of frame**,

- which changes from a world frame to a camera frame.
- We can do the view transformation with **4×4 LookAt matrix**.

*World frame에서 camera frame으로
바꾸기 위해 4x4 LookAt matrix
||
View transform*

Building LookAt Matrix as Change of Frame

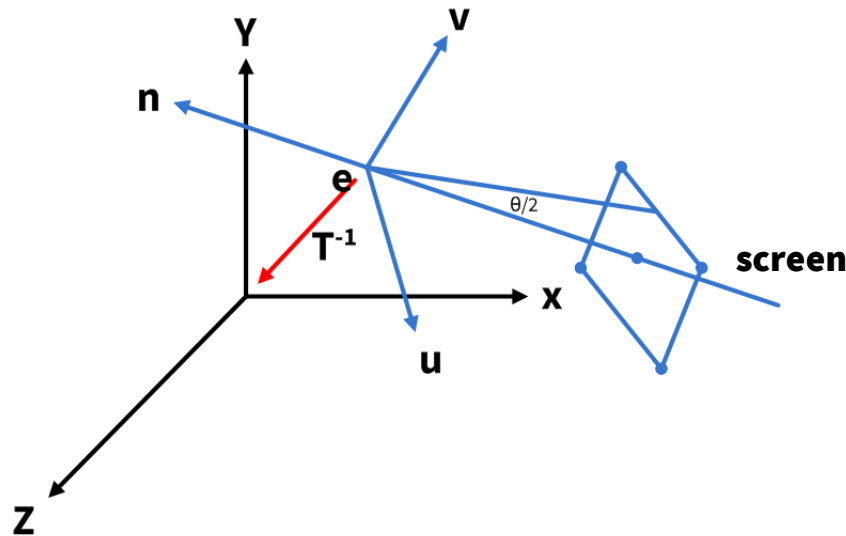
- **First step:**

- Translation to the negative eye position:

*Camera → World 으로 translation
frame frame*

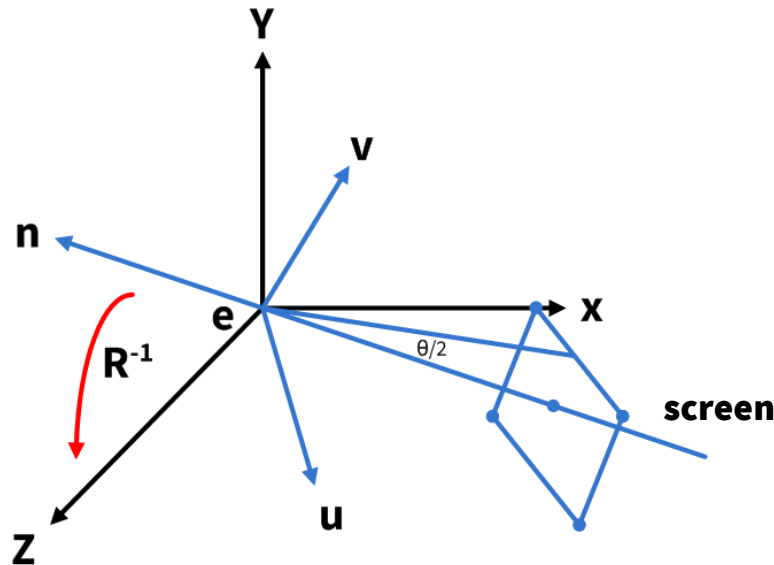
*원점을 같은 basis
vector만 가름*

$$T(-eye) = \begin{bmatrix} 1 & 0 & 0 & -eye.x \\ 0 & 1 & 0 & -eye.y \\ 0 & 0 & 1 & -eye.z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Building LookAt Matrix as Change of Frame

- **Second step: change of coordinate system**
 - Change of **orthonormal basis** = rotation matrix

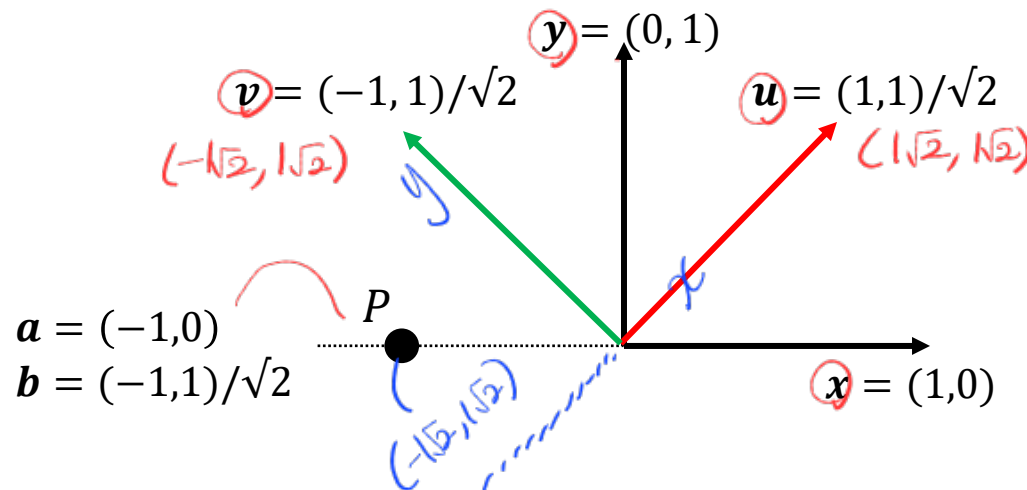


Building LookAt Matrix as Change of Frame

- **Second step: change of coordinate system**

- Intuition of the change of coordinate system in 2D
 - A black point P below can be represented in $\mathbf{a} = (-1, 0)$ and $\mathbf{b} = (-1, 1)/\sqrt{2}$ with respect to $\{x, y\}$ and $\{u, v\}$, respectively.
 - We can obtain $\mathbf{b}$ from $\mathbf{a}$ using 2×2 matrix $\mathbf{R}$, which write $(\mathbf{u}, \mathbf{v})$ in rows:

$$\mathbf{R}\mathbf{a} = \begin{bmatrix} u_1 & u_2 \\ v_1 & v_2 \end{bmatrix} \mathbf{a} = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix} \begin{bmatrix} -1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} = \mathbf{b}$$



Building LookAt Matrix as Change of Frame

- **Second step: change of coordinate system**

- Given the basis of world coordinate system, x, y, z , the eye-coordinate basis vectors can be :

$$u = u_1x + u_2y + u_3z$$

$$v = v_1x + v_2y + v_3z$$

$$n = n_1x + n_2y + n_3z$$

- Then, the relationship can be reformulated as:

$$[u \quad v \quad n] = [x \quad y \quad z] \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \begin{bmatrix} n_1 \\ n_2 \\ n_3 \end{bmatrix}$$

$u = u_1x + u_2y + u_3z$

Building LookAt Matrix as Change of Frame

- **Second step: change of coordinate system**

- Two representations $\mathbf{a}$ and $\mathbf{b}$ with respect to (x, y, z) and (u, v, n) can be related as:

$$a_1\mathbf{x} + a_2\mathbf{y} + a_3\mathbf{z} = b_1\mathbf{u} + b_2\mathbf{v} + b_3\mathbf{n}$$

$$[x \ y \ z] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [u \ v \ n] \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = [x \ y \ z] \begin{bmatrix} u_1 & v_1 & n_1 \\ u_2 & v_2 & n_2 \\ u_3 & v_3 & n_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- When we remove $[x \ y \ z]$ from LHS and RHS,

$$\mathbf{a} = \begin{bmatrix} u_1 & v_1 & n_1 \\ u_2 & v_2 & n_2 \\ u_3 & v_3 & n_3 \end{bmatrix} \mathbf{b}$$

Building LookAt Matrix as Change of Frame

- **Second step: change of coordinate system**

$$\mathbf{a} = \mathbf{R}^T \mathbf{b}, \quad \text{where} \quad \mathbf{R} = \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 & \mathbf{u}_3 \\ \mathbf{v}_1 & \mathbf{v}_2 & \mathbf{v}_3 \\ \mathbf{n}_1 & \mathbf{n}_2 & \mathbf{n}_3 \end{bmatrix}$$

- Since the transpose of a matrix of the orthonormal basis (i.e., rotation matrix) is equivalent to the inverse,
$$(\mathbf{R}^T)^{-1} \mathbf{a} = (\mathbf{R}^T)^T \mathbf{a} = \mathbf{R} \mathbf{a} = \mathbf{b}$$
- Hence, matrix $\mathbf{R}$ transforms the world-coordinate representation $\mathbf{a}$ to the eye-coordinate representation $\mathbf{b}$.

Building LookAt Matrix as Change of Frame

- **The final 4×4 LookAt (view) matrix**

- By combining the translation and rotation matrices, the final view matrix becomes:

$$\mathbf{RT}(-eye) = \begin{bmatrix} u_1 & u_2 & u_3 & 0 \\ v_1 & v_2 & v_3 & 0 \\ n_1 & n_2 & n_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -eye.x \\ 0 & 1 & 0 & -eye.y \\ 0 & 0 & 1 & -eye.z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Exercises

Exercise: Change of Frames

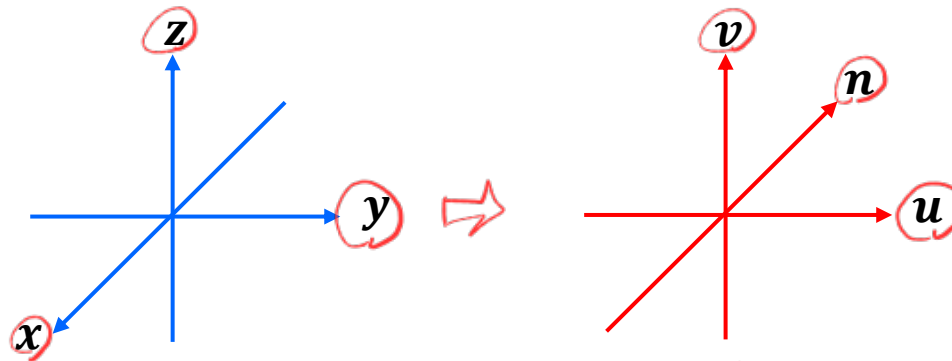
바뀐 것은 basis vector를 가르르 쓰면 된다.

• Do the following on your own.

- Derive a matrix that changes a representation of a frame $\{0, x, y, z\}$ to that of a frame $\{0, u, v, n\}$. 0 is the origin shared between the two frames.
- Hint: write $\{u, v, n\}$ in terms of $\{x, y, z\}$. Then, do as you learn.

rotation만 하면 될

origin



① u, v, n 을 x, y, z 로
linear combination을 쓴다

→

$$\begin{aligned} u &= 0x + 1y + 0z \\ v &= 0x + 0y + 1z \\ n &= -x + 0y + 0z \end{aligned}$$

② 가르르 쓴다

→

② $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 0 & 0 \end{bmatrix}$ → 여기서 xyz representation을
찾으면, u, v, n 의 representation을 바꾼다